

Deep Learning Approaches on Tissue Segmentation and Nuclei Classification

Student No. 2908478: Tissue Segmentation Task

Student No. 2574221: Nuclei Classification Task

1. Introduction

This report investigates different deep learning methods in Computer Vision for histopathological image analysis. Here, two tasks are completed: tissue segmentation with custom U-Net models and nuclei classification with convolutional neural networks. The complete pipeline will be conducted, from data pre-processing to model training and evaluation.

2. Datasets & Preprocessing

This report uses the PUMA data set as the primary data source. The segmentation task is performed using the Melanoma Tissue dataset, while the classification task is performed using the Melanoma Nuclei dataset.

2.1. Segmentation Dataset

The segmentation data set consists of 205 images with a resolution of 1024 x 1024 pixels, each paired with the corresponding ground truth mask. Data were split into training, validation, and testing sets with an approximate ratio of 80:10:10.

2.1.1. Image Annotation

The annotations were provided in GeoJSON formats, where each image contained either polygon or multipolygons representations of tissue regions. These annotations were converted into pixel-wise masks, where each pixel was assigned a class label: 1 for tissue tumor, 2 for tissue stroma, and 0 for others.

2.1.2. Image & Mask Preprocessing

The RGB values in the image is normalized into a scale of 0 to 1. Due to the memory constraints and computation cost, all images are resized to 256x256 pixels using bilinear interpolation. The mask representations also handled the downsizing accordingly with scaling factor (H_{new}/H_{ori} and W_{new}/W_{ori}). Although patch-based approaches can be performed to retain the full resolution and increase the number of training samples, they often fail to capture the global tissue context, which is important to understand how tissues are spatially structured across the whole images. This information is particularly important in

region-level tissue segmentation, where spatial patterns are meaningful [8]. Therefore, resizing the full image was preferred over patching in this work.

2.1.3. Data Augmentation

To increase the variation of the training sets, data augmentation was implemented using Albumentations library to apply random transformations to the training images. The transformations include, vertical and horizontal flipping ($p = 0.5$), 90-degree rotations ($p = 0.5$), brightness and contrast adjustments ($p = 0.4$), and color properties shifts ($p = 0.3$). These augmentation can help improve the model's ability to generalize and reduce the risk of overfitting. This augmentation is not applied to the validation and test sets.

2.2. Classification Dataset

2.2.1. Preprocessing Files

The train and validation data sets were given as a combination of images with a resolution of 1024x1024 pixels along with a GEOJSON file for their coordinates within the image and the classification type for each coordinate system. Each image and GEOJSON pair had multiple Polygon and Multipolygon coordinate systems that pointed to multiple samples. Leading to extracting the coordinates and annotation as a pair, using zero-padding whenever close to the border of the image.

2.2.2. Test Dataset

The test data set of the classification model follows the structure of a 100x100 pixels image along with the classification in the name of the NPY file.

2.2.3. Train, Contrastive and Validation Dataset

The classification training dataset consists of 15000 images with a resolution of 100x100 pixels, each paired with a corresponding annotation. Each classification had 5000 samples of the 15000. The training data set was first randomized and then split into two separate datasets, training and contrastive, preventing data leakage between the two. The validation data set consists of 2100 images with a resolution of 100x100 pixels, 700 samples for each class, along with randomization. Using patches here as global context

does not matter as much to specific classifications of individual tissue.

2.2.4. Image Augmentation

Data augmentation is applied for both training data and contrastive data sets in order to help the models distinguish between useful features and spatial noise. Transformations such as random flips or changing brightness along with contrast are used.

3. Network Design

3.1. Segmentation Network Architecture

3.1.1. Custom U-Net Model

We adopt the U-Net architecture for the tissue segmentation tasks [6]. The model consists of an encoder-decoder structure with skip connections between the encoders and decoders, as shown in Figure 1a. The encoders progressively down-sample the image while extracting hierarchical patterns of the tissue through convolution and max-pooling layers. Early layers help capture local spatial details, such as edges and corners, while the later ones recognize higher-level semantic information. The decoders, on the other hand, attempt to symmetrically up-sample and reconstruct the original image. With the use of skip connections, decoders also obtain the detailed features from earlier encoders, which brought refined spatial details of the image that might be lost during the down-samples.

The model takes in the input of shape (B, 3, H, W), where B denotes the batch size, 3 refers to the three RGB channels, and H and W represents the spatial dimension of each input image. Our UNet uses 3 encoder depths with base channel width of 64, resulting in channel dimensions of 64, 128, and 256 at later stages, followed by a bottleneck of 512 channels. Three encoder stages were chosen instead of the original four, as four stages would produce the image resolution of 16 x 16 at the bottleneck with the 256 x 256 input size, which could over-compress the spatial details and complicate categorization tasks on the massive tissue like tumor and stroma. Each convolutional block composes of two 3x3 convolutions with Group Normalization and ReLU activation:

$$\text{ConvBlock}(x) = \text{ReLU}(\text{GN}(\text{Conv}(\text{ReLU}(\text{GN}(\text{Conv}(x))))))$$

Here, Group Normalization is used instead of Batch Normalization, as it is independent of the batch size and produce more stable normalization for the small batch size required in this case due to the small number of training sets [10]. The three decoders use transposed convolutions to upsample feature maps, concatenate them with the same-level skip connection, and pass through a convolutional block to process new features. Lastly, the final feature map is delivered to the last 1x1 convolution layer that maps 64 channels to 3 output classes, producing the output of

shape (B, 3, H, W). This will later be passed to a softmax activation for class probability estimation map.

3.1.2. Autoencoder Model

For task (b), the pre-training strategy is being used where the encoder blocks adopt the weights from the pre-trained autoencoder. Our autoencoder shares identical structures with the UNet explained previously except for the skip connections that got removed, meaning the decoders will only receive input from the successive layers. The architecture of the model can be shown in Figure 1b. The aim of this autoencoder is not class predictions but image reconstruction, with an assumption that, if the model understands the underlying features and patterns of the tissues to the point that it can reconstruct the almost-identical image within the pipeline, these trained encoder weights should be able to learn and extract refined details that would make segmentation task more accurate.

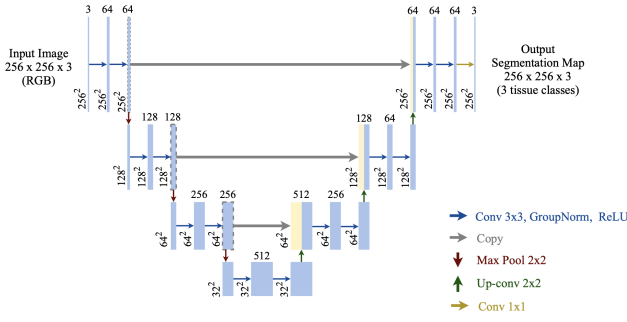
3.2. Classification Network Architecture

3.2.1. End to End Supervised Model

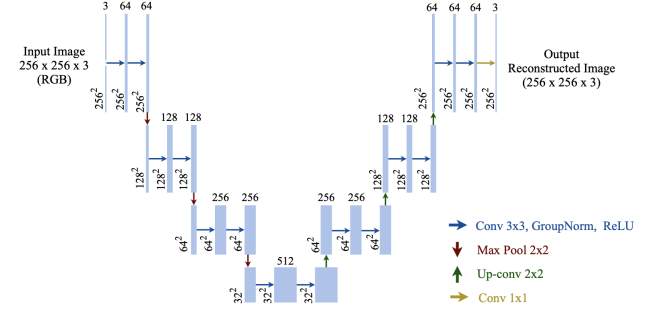
The End-to-End CNN model takes in the input of shape (64, 100, 100, 3), which is meant to represent the image dimensions (100x100 pixels), with 3 referring to the three RGB channels and batches of 64 images.

Early layers use MaxPooling to emphasize dominant features in the image, while later layers use AveragePooling to average out features and preserve the overall structure before global aggregation. We also use GlobalPooling to help reduce model complexity, and number of parameters, which can help mitigate overfitting. The reasoning for the kernel sizes is a larger kernel to capture larger spatial context in the first convolutional block and a slight decrease in kernel size for local feature extraction in middle layers (convolutional blocks 2 and 3). Batch normalization is used throughout to help stabilize the training and improve convergence during the training [2].

We use the nonlinear activation function *ReLU* to help mitigate the vanishing gradient problem, along with adding non-linearity to our neural network framework. After the Global Pooling, we move onto fully connected layers which take the features extracted in the three convolutional blocks and maps features to class probabilities. Dropout is used in the second convolutional block to help prevent overfitting and regularize the output of the model through a dropout rate of 0.25 [9], while also used before the output softmax function to help reduce reliance on any single feature. The output function is softmax as it is used for multi-class classification.



(a) U-Net architecture



(b) Autoencoder architecture for pre-training in task(b)

Figure 1. Model architecture for tissue segmentation

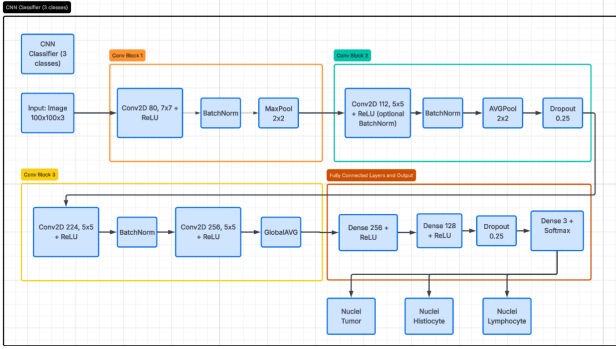


Figure 2. Architectural design of End-to-End supervised classification model

3.2.2. Contrastive Pre-Trained SimCLR Model

We adopt a simplified SimCLR architecture [1] for pre-training the encoder derived from the end to end model. SimCLR learns meaningful visual representations by maximizing agreement between two different augmentations of the same image while contrasting them against other samples in the batch, encouraging the model to learn invariant features. This part is useful for classification of nuclei, where they share similar underlying structures, but subtle differences between them.

The structure of the architecture is the combination of an encoder (our End-to-End model), a projection head, and a classification head or model. The projection head is used during contrastive pre-training so that the encoder is not directly shaped by the contrastive objective, allowing it to retain more useful features for downstream classification. The classification head essentially uses the encoder’s outputs as its features and maps learned representations to class labels from pre-training to boost accuracy.

Our implementation is a simplified version of full SimCLR [3]. In particular, it is trained in a smaller-scale setting with smaller batch sizes than the original framework, which reduces the number of negative samples available during contrastive learning. In addition, the loss uses a simplified symmetric

contrastive formulation inspired by the Normalized Temperature-scaled Cross Entropy (NT-Xent) loss used in the original SimCLR method. This loss function works by encouraging representation between two augmented versions of the same image, while pushing away other images in the batch. This is achieved by comparing positive pairs against all other images in the batch. The goal was therefore not to fully reproduce the original SimCLR training regime, but to evaluate whether contrastive pre-training could improve downstream classification performance in this dataset.

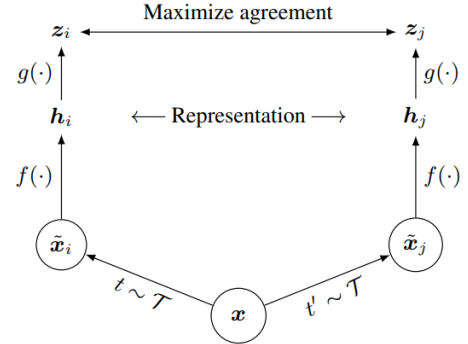


Figure 2. A simple framework for contrastive learning of visual representations. Two separate data augmentation operators are sampled from the same family of augmentations ($t \sim \mathcal{T}$ and $t' \sim \mathcal{T}$) and applied to each data example to obtain two correlated views. A base encoder network $f(\cdot)$ and a projection head $g(\cdot)$ are trained to maximize agreement using a contrastive loss. After training is completed, we throw away the projection head $g(\cdot)$ and use encoder $f(\cdot)$ and representation h for downstream tasks.

Figure 3. An explanation of the process of SimCLR framework from [1]

4. Training Procedure

4.1. Model Training for Segmentation Task

4.1.1. Loss Function

Training a segmentation model on the tissue images has one critical challenge: the dataset is heavily imbalanced, with the tumor class occupying almost 70%

of the entire pixels on average, while stroma and 'others' regions contain a much smaller proportion. To address the problem, several loss functions were evaluated (see section 6.1.1 for detailed comparisons), and the combined weighted Cross-Entropy and Dice loss with a scaling factor of 0.5 was selected for the final training, due to its robustness in handling class imbalance. The weighted Cross-Entropy term assigns higher weights to minority classes, while the Dice component directly optimizes the overlap between predicted regions and ground truth.

The weighted Cross-Entropy loss is defined as:

$$\mathcal{L}_{WCE} = - \sum_{c=1}^C w_c \sum_j t_j^c \log(p_j^c) \quad (1)$$

where w_c is the inverse-frequency weights, p_j^c is the predicted probability of pixel j belonging to class c (soft assignment), t_j^c is the ground truth mask of each pixel (hard assignment; 1 if pixel j belongs to class c , 0 otherwise), and $C = 3$ is the number of classes.

The Dice loss is computed per class and averaged among all classes:

$$\mathcal{L}_{Dice} = \frac{1}{C} \sum_{c=1}^C \frac{2 \sum_j p_j^c t_j^c + 1}{\sum_j p_j^c + \sum_j t_j^c + 1} \quad (2)$$

The smoothing variable of 1 is added to both numerator and denominator to prevent division error in case the class is absent. If the entire class is not present in the ground truth mask, the function will not include that class as part of the computation to prevent wrong rewarding.

The final loss function becomes:

$$\mathcal{L} = \alpha \mathcal{L}_{WCE} + (1 - \alpha) \mathcal{L}_{Dice} \quad (3)$$

where $\alpha = 0.5$ balances the pixel-wise classification accuracy with regional overlap. The loss is computed per batch, ensuring no single image dominates the gradient update process.

4.1.2. Task (a) U-Net Training from Scratch

The U-Net model was trained from randomly initialized weights, allowing both encoder and decoder to learn segmentation-specific features directly from the data during training.

The Adam optimizer [4] was used with the initial learning rate of 0.001. To optimize the training pipeline especially during the point close to convergence, a ReduceLROnPlateau scheduler was applied to reduce the learning rate by half if no improvement in validation dice score was found in 7 consecutive epochs. Training proceeds for a maximum of 200

epochs with a batch size of 16, which provides a sufficient stability for gradient updates within the memory constraint and data size.

To improve generalization on the limited number of training sets, random augmentation was applied to training images, as described in Section 2.1.3.

Early stopping was implemented during the training with patience of 30 epochs monitored on the improvement of validation loss to avoid overfitting. The model parameters were constantly saved at the checkpoints that report either the best validation loss or the best average dice score in the validation set at that time.

4.1.3. Task (b) Autoencoder Pre-training for Segmentation

Task (b) investigates on whether pre-training autoencoder on image reconstruction before segmentation training could improve the segmentation performance. We have a hypothesis that the pre-trained encoder can extract meaningful low-level and mid-level features of the tissue structures, which could help with the segmentation label task in the next step in decoder. The training consists of 2 phases:

Phase 1 - Autoencoder Training: The autoencoder received an unsupervised training to reconstruct similar image as the input. The model was trained with the same 163 training sets with a batch size of 16 and a total of 1000 epochs. The same augmentation pipeline as in task(a) was implemented. The aim of the training is to minimize the Mean Square Error between the original image and the reconstructed one:

$$L_{MSE} = \frac{1}{N} \sum_i (x_i - \hat{x}_i)^2 \quad (4)$$

where x_i and $\hat{x}_i$ denotes the corresponding pixel values of the original image and predicted image, respectively and N is the total number of pixels in an image. Adam optimizer was used again with the constant learning rate of 0.001. No scheduler or early stopping was used in the autoencoder training, as it was observed that the more the epochs the better the quality of the reconstructed images.

Phase 2 - Segmentation Training: The identical U-Net architecture and training pipeline as task (a) was implemented in task (b) for clear comparison. Here, the encoder weights were initialized from the autoencoder trained in phase 1. The parameters transfer can be implemented seamlessly as the two models share the same encoder architecture. The encoder is frozen during the training in phase 2 to preserve the full capability of the encoder in potentially extracting essential and structural features among compressed data that might be useful for class discrimination.

4.1.4. Evaluation Metrics

We use the macro-averaged Dice score as our main evaluation metrics, since it treats all classes equally regardless of the imbalanced class distribution.

It initially computes the per-class Dice of class j as follows:

$$\text{Dice}_j = \frac{2TP_j}{2TP_j + FP_j + FN_j} \quad (5)$$

where TP is true positive (ground truth and prediction overlap), FP is false positive, and FN is false negative.

The per-class Dices are then averaged out for the macro-averaged Dice:

$$\text{Macro-average Dice} = \frac{1}{C} \sum_{j=1}^C \text{Dice}_j \quad (6)$$

By considering both per-class Dice and macro-averaged Dice, we can evaluate the model's performance on individual classes as well as its overall performance fairly across the dataset. This can also help assess the impact of class imbalance on model training and the effectiveness of different loss function choices.

4.2. Model Training for Classification Task

4.2.1. Loss Function

The training of the classification model was done through batches of 64 samples, along with normalization of image values from $[0, 255]$ to $[0, 1]$ to have a common range, and we take the data which before would have been a straight annotation string and turn it into a one-hot encoded vector to represent class types. We also augment the images during training to help generalize, but not validation or test as we want to predict on unaltered images. The loss function categorical cross entropy was chosen as it is appropriate for multi-class classification with one-hot encoded labels and aligns with the probabilistic outputs of the softmax activation, encouraging correct and confident predictions:

$$\mathcal{L} = -\frac{1}{N} \sum_{n=1}^N \sum_{i=1}^C y_{n,i} \log(\hat{y}_{n,i})$$

Due to the nature of the class imbalance, which is discussed in the results, we implemented a `class_weight` to train the end-to-end model to help better weigh the errors of the different classes. Hyperparameter tuning came from an originally wider range of values and a slow shrinking of the ranges towards more optimal parameters that produced better results.

# Original Hyperparameter search space	# Refined Hyperparameter search space
filter1_options = [16, 32, 48, 64, 80, 96]	filter1_options = [64, 80, 96]
filter2_options = [64, 96, 128, 160, 192]	filter2_options = [112, 128, 160]
filter3_options = [128, 192, 224, 256, 384]	filter3_options = [192, 224, 256]
first_kernel_size_options = [5, 7]	first_kernel_size_options = [5, 7]
other_kernels_size_options = [3, 5]	other_kernels_size_options = [3, 5]
pool_size_options = [2, 3, 4]	pool_size_options = [2]
strides_options = [2, 3, 4]	strides_options = [2]
dense1_options = [128, 256, 512, 1024, 1536, 2048]	dense1_options = [128, 160, 224, 256]
dense2_options = [64, 128, 256, 384, 512]	dense2_options = [64, 96, 128, 160]
dropout_rate_options = [0.2, 0.3, 0.4, 0.5]	dropout_rate_options = [0.15, 0.2, 0.25, 0.3]

Figure 4. Parameter evolution to smaller ranges

For the contrastive pre-training model, the contrastive loss function based on the SimCLR framework, which essentially encourages the model to learn representations where augmented views of the same image are mapped close together, while representations of different images are pushed apart. This is achieved by using a batch-wise similarity comparison, training each sample to recognize its corresponding augmented pair among all images in the batch.

4.2.2. Task (a) Training an End-to-End Model From Scratch

The training for the end to end model was a random selection of the parameters within the specified ranges of the original and refined parameter values. For each instance of a model, we would have 90 epochs on batches of 64, providing a balance between stable gradient updates and computational efficiency. For our optimizer, we used Adam [4] with a learning rate of 0.0003. If we saw a plateau in improvement for `val_accuracy` over 12 epochs, we implemented the reducible learning rate callback to reduce our learning rate by a factor of 0.3, never falling below 1×10^{-6} . Another callback we implemented was the early stop, which if over the course of 15 epochs there is no major improvement over a certain validation performance, then we restore the best weights seen during training for validation. We then evaluate against the validation set and select the model that performed the best based on validation accuracy. If so, we replace and continue searching for a fixed number of trials period of random parameter models. Although not a full grid search (which would be computationally expensive and long), our implementation does cover many different combinations. During training, in the `fit` method, we create the batches of data, transforming our data through normalization, augmentation of the training data, and finally one-hot encoding our annotations. The class weights `{Tumor: 1.0, Histiocyte: 1.5, Lymphocyte: 1.0}` are used to help the model pay more attention to the minority class (Histiocyte) and learn to make better predictions for it. This configuration led to improved recall for the minority class, jumping by ~ 0.1 .

4.2.3. Task (b) Contrastive Model Pre-Training

The same architecture used in the end-to-end model was adopted as the encoder in the SimCLR architec-

tural design to enable a clear comparison. We used the same hyperparameters found to be effective in Task (a) and applied them to the encoder class developed. For every epoch of pre-training, we followed the SimCLR protocol of every image augmented twice for maximizing agreement and contrasted against other samples. After 35 epochs of pre-training, we then froze the encoder. The projection head is discarded after pre-training as described by the SimCLR architecture. Then we have our classifier head run on the encoder as an input and outputting classes, fine tuning the classifier head over the course of 50 epochs.

4.2.4. Evaluation Metrics

The metrics we used were accuracy, precision, and recall. Accuracy provides an overall measure of performance for the model, however, accuracy alone would not tell us much as a class imbalance may lead to skewed results. Precision measures the proportion of correctly predicted samples within each predicted class and explains how reliable the model’s positive predictions are. Recall measures the proportion of correctly identified samples within each true class, reflecting the model’s ability to detect each class. These metrics are particularly important for this dataset that contains similar yet subtly different class structures. Together, these metrics allow us to assess whether the model is biased towards or against specific classes, while also providing a more comprehensive view of its generalization performance.

5. Results

5.1. Segmentation Results

The trained U-Net model for task (a) and task (b) segmentation performance was compared against the baseline model on the test set as shown in Table 1. The baseline model contains approximately 16 times more parameters than our custom U-Net model in both tasks (125M vs 7.7M). Despite having fewer trained parameters, both models surpassed the baseline in Dice score. Autoencoder-pretrained U-Net model for task (b) achieves the highest macro-average Dice score of 0.5532, outperforming the random-initialized model from task (a) by 0.0220 point, suggesting that the encoder weights from autoencoder provide meaningful starting point for the segmentation task learning under our experimental conditions.

Table 1. Segmentation performance on the test set. Dice and IoU are reported as macro-averaged scores across classes.

Model	Params	Dice	IoU
Baseline	~125M	0.4670	-
(a) Custom U-Net	~7.7M	0.5312	0.4127
(b) AE U-Net	~7.7M	0.5532	0.4279

The evaluation of the Dice score on the test set was broken down per class, as shown in Table 2. The result is consistent with the class imbalance (as mentioned in section 4.1.1), with the tumor class achieving a significantly higher dice score relative to the other two classes. The ‘others’ class shows the lowest dice score, less than 0.25, likely due to its low pixel-wise appearance in training set. It can be seen that the autoencoder-pretrained U-Net model observes relatively higher dice score on ‘others’ class, which is the main contribution to the improvement in task (b) model’s macro-averaged Dice. This means that the pretrained weights from autoencoder could provide a more sensitive starting point for recognizing the underrepresented regions.

Table 2. Per-class Dice on the test set

Model	Others	Tumor	Stroma	Mean
(a) Custom U-Net	0.1659	0.8544	0.5732	0.5312
(b) AE U-Net	0.2222	0.8568	0.5806	0.5532

To evaluate the qualitative performance of each model, the predicted masks are plotted on the images alongside ground truth. The examples of the successful cases on the test set are displayed in Figure 5, while the failed cases are shown in Figure 6. The success cases demonstrate that both models are able to capture the structural boundaries with reasonable precision, with the pretrained model for task (b) showing a more consistent and smooth predicted textures across regions, especially for the tumor areas. However, it can be seen in Figure 6 that the models sometimes struggle to detect stroma and ‘others’ classes and mislabeled them as the dominant tumor class. This false result is likely the impact of the heavy class imbalance in the training set.

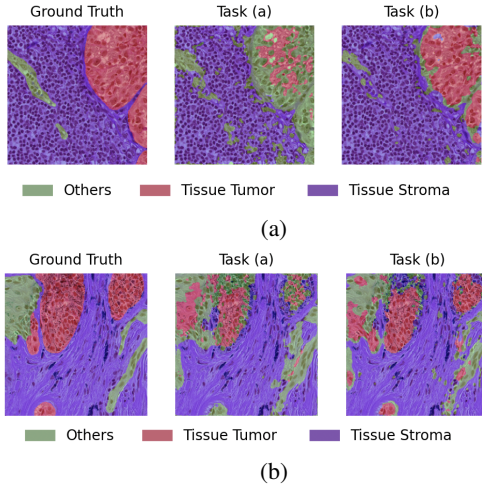


Figure 5. Segmentation prediction of each model in successful cases compared to the ground truth

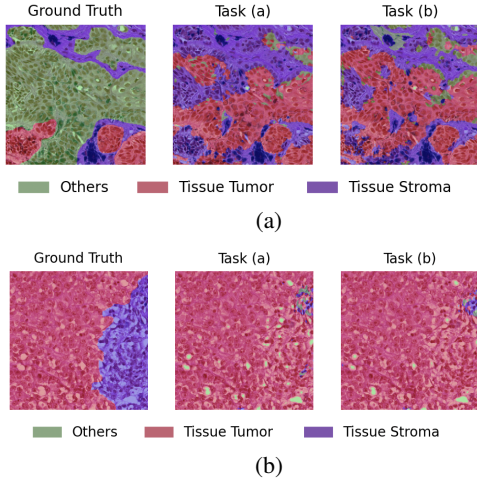


Figure 6. Segmentation prediction of each model in unsuccessful cases compared to the ground truth

5.2. Classification results

The End-to-End task (a) model achieved an accuracy of 0.7400 on the test set, outperforming both the baseline model (0.7038) and the Contrastive Pre-Trained task (b) model (0.5210). While the improvement over the baseline is modest, the end-to-end model demonstrates a substantial performance gain compared to the contrastive approach.

Table 3. Per-Class Performance Comparison Between Models

Class	Count	(a) Prec.	(a) Recall	(b) Prec.	(b) Recall
Tumor	700	0.77	0.87	0.67	0.59
Histiocyte	458	0.58	0.61	0.33	0.50
Lymphocyte	700	0.82	0.70	0.60	0.47

The Histiocyte class had the lowest precision in both models (0.58 and 0.33), and also had the lowest

recall in the end-to-end model and the second lowest recall in the pre-trained model (0.61 and 0.50). The End-to-End model consistently outperformed the Pre-trained model in every category. The test dataset also shows an uneven class distribution, with Tumor and Lymphocyte having substantially more samples than Histiocyte (700, 700, and 458, respectively).

Table 4. Macro-Averaged Precision and Recall

Model	Macro Precision	Macro Recall
End-to-End CNN	0.72	0.74
Contrastive (Pre-trained)	0.53	0.52

The end-to-end CNN achieves higher values for both metrics, indicating more balanced performance across all classes compared to the contrastive model.

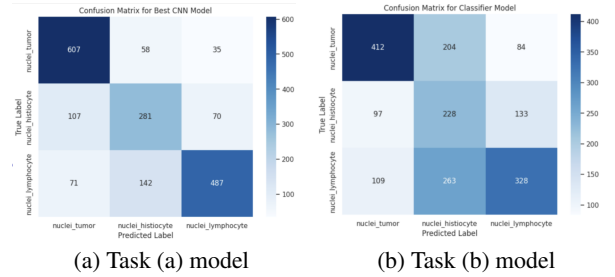


Figure 7. Confusion Matrices

The confusion matrices show generally poorer performance in the Pre-trained model compared to the End-to-End model. There is a clear pattern from both charts that the hardest class to predict was the Histiocyte class (38.6% incorrect in (a) and 50.2% incorrect in (b)). In terms of absolute error count, Lymphocyte samples account for the largest number of misclassified instances in both models (213 and 372 wrongly predicted, respectively for (a) and (b)). Overall, the end-to-end model demonstrates more accurate and balanced predictions across all three classes.

Table 5. Accuracy Across Sample Types

File Type	Count	(a) Accuracy	(b) Accuracy
Primary	1138	0.7302	0.5176
Metastatic	720	0.7556	0.5264

In both models, the metastatic data samples achieve slightly higher accuracy than the primary samples (1-2% difference). Additionally, while the End-to-End model achieves slightly higher overall accuracy on metastatic samples, the precision and recall are higher for primary samples.

Table 6. Macro Precision and Recall Across Sample Types

File Type	(a) Prec.	(a) Recall	(b) Prec.	(b) Recall
Primary	0.72	0.74	0.53	0.53
Metastatic	0.67	0.65	0.47	0.47

The silhouette score, which measures the degree of separation between clusters, is -0.02, indicating weak clustering of the data. As shown in the t-SNE visualization of the latent space (Figure 8), there is very little distinguishable clustering for the Histiocyte class, with minimal separation observed for the other classes. A high degree of overlap is present across all classes.

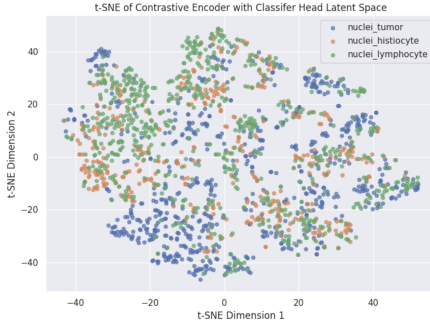


Figure 8. Latent space visualization

6. Discussion

6.1. Segmentation Task

6.1.1. Class Imbalance and Loss Function Options

The data set has a heavy class imbalance, with tumor class taking up 69.2% of total pixels across the training set, compared to 25.3% for stroma class and 5.5% for 'others' class. This could create favoritism during model training, where the model might predict most pixels as tumor class to receive higher pixel accuracy, resulting in the model failing to learn any meaningful patterns along the training.

The problem can be mitigated with the right choice of loss function that handles the class imbalance properly. To address the issue, we evaluated four loss functions: a standard Cross-Entropy (CE), a weighted Cross-Entropy with inverse-frequency class weights (WCE), a combined of weighted Cross-Entropy and Dice loss with different scaling factor (where $\alpha = x$ means the loss consists of $100x\%$ WCE and $100(1 - x)\%$ Dice loss), and a sole Dice loss. The model training with each loss function was conducted using similar settings with a constant seed for same-weight initialization. The result is shown in Table 7.

From the experiments, the standard Cross-Entropy loss, which treats all pixels equally, produces only 0.0032 Dice score on the minority 'others' class, re-

flecting how the model was heavily biased to the majority class if every class is treated equally. Weighted Cross Entropy loss improved performance by giving more importance on the inferior classes, resulting with a sharp increase in 'others' class score and a better mean Dice score. However, the improvement is limited due to its pixel-wise computation core. In contrast, the combined Cross-Entropy and Dice loss constantly outperforms the first single two, with $\alpha = 0.5$ yielding the best Dice score of 0.5312. This balance preserves the good traits of both terms: WCE ensures the stable gradients that converge quickly while Dice loss addresses the class imbalance by normalizing per-class distribution. Although Dice loss directly takes account of class imbalance, the fact that the combined loss overcome sole Dice shows that WCE helps complement the Dice term and optimize the loss function.

Despite several loss function trials, the Dice score of the 'others' class remains relatively low, and clear misclassification, especially against minority classes, can still be observed in some test samples, as shown previously in Figure 6. These quantitative and qualitative evidences demonstrate that the class imbalance issue still persists, and more advanced loss function is needed to improve the prediction of the minority class.

Table 7. Per-class Dice score comparison on validation set across different loss functions. Epoch is the number of iteration that produces the best validation macro Dice.

Loss Function	Epoch	Others	Tumour	Stroma	Mean
CE	43	0.0032	0.8669	0.5805	0.4836
WCE	31	0.0931	0.8699	0.5243	0.4958
WCE + Dice ($\alpha = 0.3$)	71	0.1197	0.8736	0.5955	0.5296
WCE + Dice ($\alpha = 0.5$)	73	0.1659	0.8544	0.5732	0.5312
WCE + Dice ($\alpha = 0.7$)	38	0.1103	0.8619	0.5630	0.5117
Dice	54	0.1392	0.8443	0.5590	0.5142

6.1.2. Task (a) vs Task (b) Comparison

The autoencoder-pretrained U-Net model for task (b) outperformed the normal one for task (a) by 0.0220 in macro-averaged Dice score. The finding is consistent with the hypothesis that pretrained weights can improve segmentation learning by providing more meaningful initialization than a random one. The encoder that has learned the reconstruction task might have developed features extraction and pattern recognition, such as spatial structure and anatomical context, which could be useful and applicable to the discriminative features learning required for the segmentation task.

Considering the per-class Dice score in Table 2, the improvement is mainly driven by the 'others' class Dice, while the tumor and stroma Dice remain simi-

lar between the two models. Since the 'others' class is the combination of every tissue aside from tumor and stroma, the structure can be more visually inconsistent. Hence, by being able to learn general structure of the image, pretraining could help the model better understand how to distinguish ambiguous regions that do not clearly belong to either tumor or stroma class. This is especially beneficial compared to the randomly initialized model, which may struggle to detect the non-structural pattern of the class more, making it harder for the model in task (a) to accurately recognize the 'others' region.

6.1.3. Parameter Efficiency

Our U-Net model, with approximately 7.7 million parameters, outperforms the baseline model, with roughly 125 million parameters. This could be due to the mismatch of the baseline model between the number of training data sets and the learnable parameters. With only 163 images in the training set, having a substantially large number of learnable parameters could backfire and be more prone to overfitting, making the model generalize worse in the unseen data and produce lower Dice score on test set.

6.2. Classification Task

6.2.1. Latent Space Evaluation

For the contrastive latent space of the pretrained model, there was much overlap between the classes. Using a silhouette score [7] from the TensorFlow library, which ranges from the values of $[-1, 1]$ for the strength of the separation of clusters, we receive a -0.02 , which signals very weak separation. We believe that this weakness is in part due to the encoder freezing and being unable to learn any more important information beyond pre-training. As unfreezing part of the encoder for further training allows for a general increase in accuracy from $\sim .5$ to $\sim .6$.

The test set for the classification tasks was heavily imbalanced in the class types presented. As seen in Table 3, there are approximately $1.5\times$ as many samples for the Tumor and Lymphocyte classes compared to the Histiocyte class in the test data. This uneven distribution likely biases the model toward the majority classes, resulting in reduced performance on the underrepresented Histiocyte class. The CNN model seems to generally misclassify approximately one fourth of Histiocyte samples as Tumors, and the Pre-trained model effectively struggles to distinguish Histiocytes (approximately 50% of the Histiocytes are not predicted as Histiocytes). Furthermore, the latent space data shows poor clustering for Histiocyte, indicating poor separability and therefore higher misclassifications (Figure 8). This is shown in the evalua-

tion metrics, where the Histiocyte class exhibits lower recall because a large proportion of true Histiocyte samples are misclassified as Tumors or Lymphocytes (Table 3). This indicates that the model struggles to correctly identify this class. Additionally, misclassifications into other classes not only reduce precision for those classes, but also counted as false positives in those classes. At the very least, there appears to be some general clustering for the tumor and lymphocyte classes, however, not strong enough to be substantial.

6.2.2. Task (a) vs Task (b) Comparison

The performance difference between the pre-trained model and the end to end model does not support the hypothesis that pre-training on large unlabeled datasets is always useful for small annotated datasets. One likely factor affecting the performance of the pre-trained model is the freezing of the encoder, which prevents the encoder from learning further task-specific features after the pre-training phase. Additionally, SimCLR learns representations by maximizing agreement between augmented views of the same image and reducing agreement to other images, rather than explicitly optimizing for class separation. As a result, the learned features may not align well with the downstream classification task. The highly detailed nature of medical images also requires fine-grained feature extraction, which the frozen encoder cannot learn. The inability of the frozen encoder to refine its representations likely contributes to its poorer performance, particularly for more challenging classes such as Histiocytes.

6.2.3. Metastatic and Primary Samples

Interestingly, when comparing performance for primary and metastatic samples, we observe similar trends across the two models. For both models, there seems to be a real difference, albeit small, between the performance on the primary data and the metastatic (1-2% accuracy difference in favor of metastatic). This is notable, as one might expect primary samples to have more consistent structural characteristics compared to the metastatic spread. However, there are studies that suggest the opposite, that metastatic tumors often have reduced heterogeneity and increased clonality compared to primary tumors [5]. These properties may contribute to easier feature extraction for metastatic data samples. The evaluation metrics also point to a difference between the samples. Specifically, primary samples exhibit slightly higher precision and recall for both models, indicating that the model is both more accurate and more consistent in identifying these samples. This suggests that, despite prior expectations, the primary samples in this dataset may exhibit more consistent or learn-

able feature representations for the model.

6.2.4. Parameter Comparison

The number of trainable parameters in the End-to-End model is much greater than those of the Pre-trained model, as the Encoder in the latter is frozen and therefore has most of the parameters untrainable. This limits the ability for the Pre-trained model to adapt and classify the samples, whereas the End-to-End model can learn and adapt to the features because almost all of the parameters are trainable. This difference in adaptability to the classification task explains the substantial gap between the accuracy of the models $\sim .74$ versus $\sim .52$. This problem has less to do with the size of the classifier head, but rather the frozen encoder. Since the encoder is not adapted to the classification task, the model is constrained by the quality of the learned representations, and even with a larger classifier it would be unlikely to see massive improvement.

Table 8. Model Parameter Comparison

Model	Perform.	Trainable	Not-Trainable	Total
Baseline	.7038	$\sim 5\text{M}$	–	$\sim 5\text{M}$
End-to-End	.7400	$\sim 2.4\text{M}$	~ 800	$\sim 2.4\text{M}$
Pre-trained	.5210	$\sim 8.5\text{K}$	$\sim 2.3\text{M}$	$\sim 2.31\text{M}$

Interestingly, the baseline model contains approximately twice the number of trainable parameters compared to the end-to-end CNN, yet achieves lower accuracy (0.7038 vs 0.74). This demonstrates that increased model capacity does not necessarily lead to improved performance. Instead, it suggests that the architecture of the end-to-end model is more efficient, allowing it to learn more relevant features with fewer parameters. In contrast, the baseline model may contain redundant or less effective parameters, leading to diminished performance despite its larger size.

References

- [1] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 1597–1607, 2020. [3](#)
- [2] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 448–456, 2015. [2](#)
- [3] Keras. Semi-supervised image classification using contrastive pretraining with simclr. https://keras.io/examples/vision/semisupervised_simclr/, 2023. Accessed April 5, 2026. [3](#)
- [4] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015. [4, 5](#)
- [5] Francisco Martínez-Jiménez, Ali Movasati, Sascha Remy Brunner, Luan Nguyen, Peter Priestley, Edwin Cuppen, and Arne Van Hoeck. Pan-cancer whole-genome comparison of primary and metastatic solid tumours. *Nature*, 618(7964): 333–341, 2023. [9](#)
- [6] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI 2015)*, pages 234–241. Springer, 2015. [2](#)
- [7] Peter J. Rousseeuw. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987. [9](#)
- [8] Rüdiger Schmitz et al. Multi-scale fully convolutional neural networks for histopathology image segmentation: From nuclear aberrations to the global tissue architecture. *Medical Image Analysis*, 70: 101996, 2021. [1](#)
- [9] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15: 1929–1958, 2014. [2](#)
- [10] Yuxin Wu and Kaiming He. Group normalization. In *Computer Vision – ECCV 2018*, pages 3–19. Springer, 2018. [2](#)